

# **Manual técnico** **Sistema de GEstión de Flota de** **Vehículos Eléctricos** **(SIGEFVE)**

**Maestría en Sistemas Computacionales**

**Materia:**

Tecnologías de Programación

**Catedrático:**

Dr. Ulises Juárez Martínez

**Alumnos:**

Aldo Iván López Vivanco

Jessica Hernández Mayahua

Coré Husáí Martínez Belmonte

## Tabla de contenido

|                                     |           |
|-------------------------------------|-----------|
| <b>Tabla de contenido</b>           | <b>2</b>  |
| <b>Introducción</b>                 | <b>4</b>  |
| <b>Microservicio de Java</b>        | <b>5</b>  |
| Descripción                         | 5         |
| Organización de archivos            | 5         |
| Stack tecnológico                   | 6         |
| Modelo de datos                     | 6         |
| Tabla: vehiculos                    | 6         |
| Tabla: telemetria                   | 6         |
| Tabla: rutas                        | 7         |
| Tabla: entregas                     | 7         |
| Especificación de la API RESTful    | 7         |
| Vehículos                           | 7         |
| GET /vehiculos                      | 7         |
| GET /vehiculos/:id                  | 8         |
| GET /vehiculos?estado=DISPONIBLE    | 8         |
| POST /vehiculos                     | 8         |
| PUT /vehiculos/:id                  | 8         |
| PUT /vehiculos/:id/estado           | 8         |
| DELETE /vehiculos/:id               | 9         |
| Telemetría                          | 9         |
| GET /telemetria/vehiculo/:id        | 9         |
| GET /telemetria/vehiculo/:id/ultima | 9         |
| POST /telemetria                    | 9         |
| Rutas                               | 9         |
| GET /rutas                          | 9         |
| GET /rutas/:id                      | 9         |
| GET /rutas/:id/entregas             | 10        |
| POST /rutas                         | 10        |
| POST /rutas/:id/entregas            | 10        |
| PUT /rutas/:id/asignar              | 10        |
| PUT /rutas/:id/completar            | 10        |
| Health Check                        | 11        |
| GET /health                         | 11        |
| Simulador de telemetría             | 11        |
| <b>Microservicio de Python</b>      | <b>11</b> |
| Descripción                         | 11        |

|                                                |           |
|------------------------------------------------|-----------|
| Alcance                                        | 11        |
| Organización de archivos                       | 12        |
| Stack tecnológico                              | 12        |
| Diagrama de clases                             | 12        |
| Patrones de diseño implementados               | 13        |
| Observer (ObservadorTelemetry & GestorEventos) | 13        |
| Modelo de datos                                | 13        |
| Especificación de la API RESTful               | 14        |
| GET /alertas                                   | 15        |
| PATCH /alertas/desactivar                      | 15        |
| POST /telemetry                                | 15        |
| GET /estadisticas                              | 16        |
| GET /reporte/csv                               | 17        |
| GET /health                                    | 17        |
| <b>Reglas de negocio y lógica interna</b>      | <b>17</b> |
| Umbral de alertas (Config.py)                  | 17        |
| Flujo de procesamiento asíncrono               | 17        |
| <b>Despliegue y Ejecución</b>                  | <b>18</b> |
| <b>Microservicio de Go</b>                     | <b>18</b> |
| <b>Referencias</b>                             | <b>19</b> |

## Introducción

El Sistema de Gestión de Flota de Vehículos Eléctricos (SIGEFVE) es una solución integral diseñada bajo una arquitectura de microservicios para la administración, monitoreo y optimización de una flota de reparto urbano compuesta por vans, bicicletas y motos eléctricas.

El sistema permite la gestión en tiempo real de la telemetría (nivel de batería, ubicación, temperatura), la asignación eficiente de rutas y la detección proactiva de incidencias mecánicas o críticas mediante análisis de datos.

El sistema se compone de tres microservicios independientes e interconectados mediante API REST, cada uno desarrollado en el lenguaje más adecuado para su responsabilidad específica:

### Microservicio de API Gateway (Go)

- Responsabilidad: Actúa como el único punto de entrada al sistema, gestionando la seguridad y el enrutamiento.
- Funciones clave: Autenticación y autorización mediante JWT, rate limiting por IP, registro de logs de peticiones y balanceo de carga hacia los servicios internos.
- Tecnologías: Go, Framework Gin.

### Microservicio de gestión de flota (Java)

- Responsabilidad: Núcleo operativo del sistema. Maneja la persistencia transaccional de los vehículos y la ingesta de telemetría en alta frecuencia.
- Funciones clave: CRUD de vehículos, registro de telemetría (cada 15s), gestión de estados operativos (Disponible, En Ruta, etc.) y asignación de rutas de entrega.
- Tecnologías: Java, PostgreSQL, JDBC.

### Microservicio de análisis y alertas (Python)

- Responsabilidad: Procesamiento analítico y detección de eventos.
- Funciones clave: Recepción asíncrona de datos, cálculo de estadísticas de rendimiento (eficiencia de batería, km recorridos) y generación de alertas automáticas (urgentes y preventivas).
- Tecnologías: Python, Flask, SQLite.

## Microservicio de Java

### Descripción

Módulo Java del SIGEFVE responsable de la gestión de vehículos, telemetría y rutas de entrega. Implementa un servidor HTTP básico con JDBC para conectarse a PostgreSQL.

### Organización de archivos

```
com.sigefve/
├── config/
│   └── ConfiguracionBaseDatos.java # Configuración y conexión a PostgreSQL
├── controladores/
│   ├── ControladorVehiculos.java # API REST para vehículos
│   ├── ControladorTelemetria.java # API REST para telemetría
│   └── ControladorRutas.java # API REST para rutas
├── dao/
│   ├── VehiculoDAO.java # Acceso a datos de vehículos
│   ├── TelemetriaDAO.java # Acceso a datos de telemetría
│   ├── RutaDAO.java # Acceso a datos de rutas
│   └── EntregaDAO.java # Acceso a datos de entregas
├── enums/
│   ├── EstadoVehiculo.java # DISPONIBLE, EN_RUTA, MANTENIMIENTO, CARGANDO
│   ├── TipoVehiculo.java # VAN, BICICLETA_ELECTRICA, MOTO_ELECTRICA
│   └── EstadoEntrega.java # PENDIENTE, EN_CAMINO, COMPLETADA, FALLIDA
├── modelos/
│   ├── Vehiculo.java # Clase abstracta base
│   ├── VehiculoElectrico.java # Clase abstracta con propiedades eléctricas
│   ├── Van.java # Vehículo tipo Van
│   ├── BicicletaElectrica.java # Bicicleta eléctrica
│   ├── MotoElectrica.java # Moto eléctrica
│   ├── Telemetria.java # Datos de sensores
│   ├── Ruta.java # Ruta de entregas
│   └── Entrega.java # Entrega individual
├── servicios/
│   ├── VehiculoServicio.java # Lógica de negocio de vehículos
│   ├── TelemetriaServicio.java # Lógica de negocio de telemetría
│   └── RutaServicio.java # Lógica de negocio de rutas
├── simulador/
│   └── SimuladorTelemetria.java # Simulador de telemetría cada 15s
├── utils/
│   └── InicializadorDatos.java # Script para datos de prueba
```

└─ ServidorHTTP.java

# Servidor HTTP principal

## Stack tecnológico

- Java 17 (LTS)
- Maven para gestión de dependencias
- PostgreSQL 15 como base de datos
- JDBC para acceso a datos
- Gson para serialización JSON
- HttpServer (java.net) para servidor HTTP

## Configuración del servicio en Docker

Dentro del archivo docker-compose.yml, este microservicio tiene la siguiente configuración por defecto:

- Hostname: sigefve\_java
- Puerto interno: 8585
- Puerto público: 8585

Con el hostname y el puerto interno, los servicios del mismo contenedor serán capaces de comunicarse, y con el puerto público es posible acceder desde fuera del contenedor.

## Modelo de datos

### *Tabla: vehiculos*

```
CREATE TABLE vehiculos (
  id SERIAL PRIMARY KEY,
  placa VARCHAR(20) UNIQUE NOT NULL,
  modelo VARCHAR(100) NOT NULL,
  anio INTEGER NOT NULL,
  tipo VARCHAR(50) NOT NULL,
  estado VARCHAR(50) NOT NULL,
  capacidad_bateria DOUBLE PRECISION,
  autonomia_maxima DOUBLE PRECISION,
  consumo_promedio DOUBLE PRECISION,
  capacidad_carga DOUBLE PRECISION,
  numero_asientos INTEGER,
  tiene_canasta_extra BOOLEAN,
  tiene_top_case BOOLEAN,
  kilometraje_total DOUBLE PRECISION DEFAULT 0,
  fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  ultima_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

**Tabla: telemetria**

```
CREATE TABLE telemetria (
  id SERIAL PRIMARY KEY,
  vehiculo_id INTEGER NOT NULL,
  nivel_bateria DOUBLE PRECISION NOT NULL,
  latitud DOUBLE PRECISION NOT NULL,
  longitud DOUBLE PRECISION NOT NULL,
  temperatura_motor DOUBLE PRECISION NOT NULL,
  velocidad_actual DOUBLE PRECISION NOT NULL,
  kilometraje_actual DOUBLE PRECISION NOT NULL,
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (vehiculo_id) REFERENCES vehiculos(id) ON DELETE CASCADE
);
```

**Tabla: rutas**

```
CREATE TABLE rutas (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(200) NOT NULL,
  vehiculo_id INTEGER,
  distancia_total DOUBLE PRECISION NOT NULL,
  numero_entregas INTEGER DEFAULT 0,
  fecha_inicio TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  fecha_fin TIMESTAMP,
  completada BOOLEAN DEFAULT FALSE,
  FOREIGN KEY (vehiculo_id) REFERENCES vehiculos(id) ON DELETE SET NULL
);
```

**Tabla: entregas**

```
CREATE TABLE entregas (
  id SERIAL PRIMARY KEY,
  ruta_id INTEGER NOT NULL,
  direccion_destino VARCHAR(300) NOT NULL,
  latitud DOUBLE PRECISION NOT NULL,
  longitud DOUBLE PRECISION NOT NULL,
  descripcion_paquete TEXT,
  peso_kg DOUBLE PRECISION NOT NULL,
  estado VARCHAR(50) NOT NULL,
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  fecha_completada TIMESTAMP,
  notas_entrega TEXT,
  FOREIGN KEY (ruta_id) REFERENCES rutas(id) ON DELETE CASCADE
);
```

);

## Especificación de la API RESTful

### *Vehículos*

#### **GET /vehiculos**

Obtener todos los vehículos

Response: 200 OK con array de vehículos

#### **GET /vehiculos/:id**

Obtener un vehículo por ID

Response: 200 OK con vehículo o 404 Not Found

#### **GET /vehiculos?estado=DISPONIBLE**

Filtrar vehículos por estado

Response: 200 OK con array de vehículos

#### **POST /vehiculos**

Crear un nuevo vehículo

Body ejemplo:

```
{
  "tipo": "VAN",
  "placa": "VAN-001",
  "modelo": "Mercedes eSprinter",
  "anio": 2023,
  "capacidadBateria": 90.0,
  "autonomiaMaxima": 150.0,
  "capacidadCarga": 1500.0,
  "numeroAsientos": 3
}
```

#### **PUT /vehiculos/:id**

Actualizar un vehículo completo

Body: Mismo formato que POST

Response: 200 OK



**PUT /vehiculos/:id/estado**

Cambiar solo el estado de un vehículo

Body:

```
{  
  "estado": "EN_RUTA"  
}
```

**DELETE /vehiculos/:id**

Eliminar un vehículo

Response: 200 OK o 404 Not Found

***Telemetría*****GET /telemetria/vehiculo/:id**

Obtener historial de telemetría de un vehículo

Query params: ?limite=100 (opcional, default: 100)

Response: 200 OK con array de telemetría

**GET /telemetria/vehiculo/:id/ultima**

Obtener la última telemetría de un vehículo

Response: 200 OK con telemetría o 404 Not Found

**POST /telemetria**

Registrar nueva telemetría (normalmente usado por el simulador)

Body:

```
{  
  "vehiculoid": 1,  
  "nivelBateria": 75.5,  
  "latitud": 20.5288,  
  "longitud": -100.8157,  
  "temperaturaMotor": 45.2,  
  "velocidadActual": 60.0,  
  "kilometrajeActual": 1250.5  
}
```

## **Rutas**

### **GET /rutas**

Obtener todas las rutas activas (no completadas)

Response: 200 OK con array de rutas

### **GET /rutas/:id**

Obtener una ruta por ID (incluye entregas)

Response: 200 OK con ruta o 404 Not Found

### **GET /rutas/:id/entregas**

Obtener entregas de una ruta específica

Response: 200 OK con array de entregas

### **POST /rutas**

Crear una nueva ruta

Body:

```
{  
  "nombre": "Entregas Zona Centro",  
  "distanciaTotal": 15.5,  
  "vehiculoid": 1  
}
```

### **POST /rutas/:id/entregas**

Agregar una entrega a una ruta

Body:

```
{  
  "direccionDestino": "Av. Juárez 123",  
  "latitud": 20.5288,  
  "longitud": -100.8157,  
  "descripcionPaquete": "Documentos importantes",  
  "pesoKg": 2.5  
}
```

### **PUT /rutas/:id/asignar**

Asignar un vehículo a una ruta

Body:

```
{
  "vehiculoid": 1
}
```

Nota: Cambia automáticamente el estado del vehículo a EN\_RUTA

### **PUT /rutas/:id/completar**

Marcar una ruta como completada

Response: 200 OK

Nota: Cambia automáticamente el estado del vehículo a DISPONIBLE

### **Health Check**

#### **GET /health**

Verificar estado del servicio

Response: 200 OK

```
{
  "status": "OK",
  "servicio": "SIGEFVE-Java"
}
```

### **Simulador de telemetría**

El simulador genera automáticamente datos de telemetría cada 15 segundos para todos los vehículos:

- Vehículos EN\_RUTA: Consumen batería, aumentan kilometraje, se mueven en el mapa
- Vehículos CARGANDO: Recuperan batería, permanecen estáticos
- Vehículos MANTENIMIENTO: Datos estáticos
- Vehículos DISPONIBLE: Datos estables con carga lenta

El simulador considera:

- Velocidad máxima según tipo de vehículo
- Consumo de batería realista
- Temperatura del motor variable
- Movimiento geográfico dentro de Celaya, GTO

## **Microservicio de Python**

## Descripción

En esta sección se describe la arquitectura, diseño e implementación del microservicio de análisis y alertas desarrollado en Python. Este componente es crítico dentro del ecosistema SIGEFVE, encargado del procesamiento asíncrono de telemetría, la detección de anomalías en tiempo real y la generación de reportes estadísticos de rendimiento de la flota.

## Alcance

El microservicio tiene las siguientes responsabilidades:

- Ingesta de datos: Recepción de telemetría proveniente del microservicio de Java.
- Procesamiento asíncrono: Implementación de colas y threading para no bloquear el hilo principal de ejecución HTTP.
- Sistema de alertas: Evaluación de reglas de negocio (batería, temperatura, mantenimiento) y persistencia polimórfica de alertas.
- Exposición de datos: API RESTful documentada con Flasgger (Swagger UI) para consulta de estadísticas y gestión de alertas.

## Organización de archivos

```
python/
├── Abstractas/
│   ├── Notificable.py           # Clase abstracta para notificar
│   └── Registrable.py          # Clase abstracta para registrar
├── Alertas/
│   ├── Alerta.py                # Clase base de alertas
│   ├── AlertaMantenimiento.py   # Alerta de mantenimiento
│   └── AlertaUrgente.py         # Alerta urgente
├── Eventos/
│   ├── GestorEventos.py         # Threading (Subject)
│   └── ObservadorTelemetria.py  # Observador
├── Procesamiento/
│   ├── AnalizadorRendimiento.py # Análisis de rendimiento
│   └── ProcesadorEstadisticas.py # Clase abstracta para estadísticas
├── app.py                       # Punto de arranque y servidor Flask
└── Config.py                    # Configuración global (umbrales, base de datos)
```

## Stack tecnológico

- Lenguaje: Python 3.12
- Framework Web: Flask
- Documentación API: Flasgger
- Base de datos: SQLite 3 (Modo thread-safe configurado)

- Concurrencia: Biblioteca estándar threading y queue
- Comunicación Externa: requests para consumo de servicios Java.

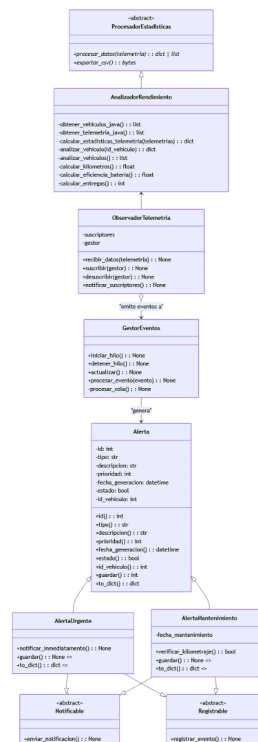
## Configuración del servicio en Docker

Dentro del archivo docker-compose.yml, este microservicio tiene la siguiente configuración por defecto:

- Hostname: sigefve-python
- Puerto interno: 5001
- Puerto público: 8822

## Diagrama de clases

El diseño del sistema sigue un enfoque orientado a objetos, utilizando patrones de diseño como Observer para la telemetría y herencia múltiple para la categorización de alertas.



## Patrones de diseño implementados

### Observer (*ObservadorTelemetria & GestorEventos*)

Desacopla la recepción de datos HTTP del procesamiento lógico. ObservadorTelemetria actúa como Subject y GestorEventos como Observer.

## Modelo de datos

La persistencia se maneja mediante SQLite. El esquema utiliza una estrategia de "Tabla por Tipo" (Table-per-Type inheritance concept) simplificada.

Tabla: alerta

Almacena los atributos base de cualquier alerta.

| Campo            | Tipo       | Descripción                                        |
|------------------|------------|----------------------------------------------------|
| id               | INTEGER PK | Identificador único autoincremental.               |
| descripcion      | TEXT       | Descripción legible del evento.                    |
| prioridad        | INTEGER    | Nivel de severidad (1: Alta, 3: Baja).             |
| fecha_generacion | TEXT       | Timestamp con la fecha de generación de la alerta. |
| estado           | INTEGER    | 1 (Activa) o 0 (Inactiva/Resuelta).                |
| id_vehiculo      | INTEGER    | FK lógica hacia el microservicio Java.             |

Tabla: alerta\_mantenimiento

Extensión para alertas de mantenimiento preventivo.

| Campo         | Tipo       | Descripción                             |
|---------------|------------|-----------------------------------------|
| id            | INTEGER PK | Identificador propio de la extensión.   |
| id_alerta     | INTEGER FK | Referencia a la tabla alerta.           |
| mantenimiento | DATETIME   | Fecha programada para el mantenimiento. |

Tabla: alerta\_urgente

Extensión para alertas críticas (batería, temperatura).

| Campo     | Tipo       | Descripción                           |
|-----------|------------|---------------------------------------|
| id        | INTEGER PK | Identificador propio de la extensión. |
| id_alerta | INTEGER FK | Referencia a la tabla alerta.         |
| mensaje   | TEXT       | Mensaje técnico detallado del error.  |

## Especificación de la API RESTful

La API se documenta automáticamente mediante Flasgger. A continuación se detallan los contratos de los endpoints implementados en app.py.

### *GET /alertas*

Recupera el listado de alertas activas (estado = 1) del sistema. Permite filtrado opcional por prioridad o vehículo.

Parámetros (Query Params):

prioridad (opcional, int): Filtra las alertas por nivel de prioridad específico.

id\_vehiculo (opcional, int): Filtra las alertas asociadas a un vehículo específico.

Respuesta 200:

```
{
  "alertas": [
    {
      "id": 1,
      "descripcion": "Batería baja",
      "prioridad": 1,
      "fecha_generacion": "2025-10-27T10:00:00",
      "estado": true,
      "id_vehiculo": 10,
      "tipo": "urgente",
      "mensaje": "Vehículo 10: Batería crítica (15%)"
    }
  ]
}
```

### *PATCH /alertas/desactivar*

Desactiva alertas (cambia estado a 0) basándose en criterios flexibles.

Parámetros (Query Params):

(Sin parámetros): Desactiva todas las alertas activas del sistema.

id\_vehiculo: Desactiva todas las alertas activas de ese vehículo.

id: Desactiva una alerta específica por su ID.

Respuesta 200:

```
{
  "mensaje": "Alertas del vehículo 5 desactivadas",
  "alertas_desactivadas": 3
}
```

### ***POST /telemetry***

Punto de entrada para los datos de sensores. Activa el ObservadorTelemetry para procesamiento asíncrono.

Cuerpo (JSON):

```
{
  "id_vehiculo": 5,
  "nivel_bateria": 18,
  "temperatura_motor": 65,
  "ubicacion": "19.4326,-99.1332",
  "kilometros_totales": 5050
}
```

Respuesta 200:

```
{ "mensaje": "Telemetría procesada" }
```

### ***GET /estadisticas***

Calcula métricas de rendimiento bajo demanda consultando al servicio Java.

Parámetros:

id\_vehiculo (opcional): ID para obtener estadísticas individuales.

Respuesta (200 OK):

Caso Global (Sin id\_vehiculo):

```
{
  "estadisticas": [
    {
      "id_vehiculo": 1,
      "registros_procesados": 150,
      "kilometros_totales": 1250.5,

```



```

    "eficiencia_bateria": 95.3,
    "entregas_completadas": 45
  },
  { "...": "..." }
]
}

```

Caso Individual (Con id\_vehiculo):

```

{
  "id_vehiculo": 5,
  "kilometros_totales": 1200,
  "eficiencia_bateria": 88.5,
  "entregas_completadas": 45,
  "registros_procesados": 300
}

```

### ***GET /reporte/csv***

Genera un archivo descargable con el análisis de toda la flota.

Header Response: Content-Type: text/csv

Disposition: attachment; filename=reporte\_rendimiento.csv

### ***GET /health***

Endpoint de monitoreo para verificar la disponibilidad del servicio.

Respuesta 200:

```

{
  "status": "OK",
  "servicio": "python-service"
}

```

## **Reglas de negocio y lógica interna**

### ***Umbral de alertas (Config.py)***

Las alertas se disparan automáticamente cuando la telemetría recibida infringe las siguientes reglas definidas en Config.py:

- Alerta Urgente (Batería): Nivel de batería < 20%.
- Alerta Urgente (Temperatura): Temperatura motor > 70°C.
- Alerta Mantenimiento: Kilometraje acumulado > 5000 km.

Nota: Al generarse, calcula la fecha sugerida de mantenimiento (fecha\_actual + 10 días).

### **Flujo de procesamiento asíncrono**

1. La petición llega a POST /telemetry.
2. ObservadorTelemetry recibe el diccionario de datos.
3. El observador notifica a GestorEventos.
4. GestorEventos coloca el evento en una Queue (Cola FIFO).
5. Un hilo demonio (Daemon Thread), iniciado al arrancar la app, consume la cola continuamente.
6. Si se detecta una anomalía, se instancia la clase correspondiente (AlertaUrgente o AlertaMantenimiento).
7. Se ejecutan los métodos enviar\_notificacion() (Consola) y registrar\_evento() (SQLite).

### **Despliegue y Ejecución**

El servicio está diseñado para ejecutarse en un contenedor Docker, exponiendo el puerto 5001. Para iniciar el servicio manualmente (desarrollo):

```
python app.py
```

## **Microservicio de Go**

### **Descripción**

En esta sección se describe la arquitectura, diseño e implementación del microservicio de análisis y alertas desarrollado en Python. Este componente es crítico dentro del ecosistema SIGEFVE, encargado del procesamiento asíncrono de telemetría, la detección de anomalías en tiempo real y la generación de reportes estadísticos de rendimiento de la flota.

### **Alcance**

El microservicio tiene las siguientes responsabilidades:

- Ingesta de datos: Recepción de telemetría proveniente del microservicio de Java.
- Procesamiento asíncrono: Implementación de colas y threading para no bloquear el hilo principal de ejecución HTTP.
- Sistema de alertas: Evaluación de reglas de negocio (batería, temperatura, mantenimiento) y persistencia polimórfica de alertas.
- Exposición de datos: API RESTful documentada con Flasgger (Swagger UI) para consulta de estadísticas y gestión de alertas.

### **Organización de archivos**

```
go/
├── auth/
```

|                    |                                                      |
|--------------------|------------------------------------------------------|
| └─ auth.go         | # Lógica para autenticar usuarios                    |
| └─ database/       |                                                      |
| └─ database.go     | # Conexión con la base de datos                      |
| └─ migrations.go   | # Migraciones de la base de datos                    |
| └─ queries.go      | # Consultas a la base de datos                       |
| └─ gateway/        |                                                      |
| └─ proxy.go        | # Proxy reverso para redirigir las peticiones        |
| └─ router.go       | # Rutas de los endpoints                             |
| └─ middleware/     |                                                      |
| └─ logging.go      | # Lógica para registrar las peticiones               |
| └─ ratelimiting.go | # Lógica para limitar el límite de peticiones por IP |
| └─ models/         |                                                      |
| └─ models.go       | # Archivo con los modelos del microservicio          |
| └─ go.mod          | # Dependencias del microservicio                     |
| └─ go.sum          | # Revisión de dependencias                           |
| └─ main.go         | # Punto de arranque                                  |

## Stack tecnológico

- Lenguaje: Go 1.25.4
- Framework: Gin
- Base de datos: PostgreSQL

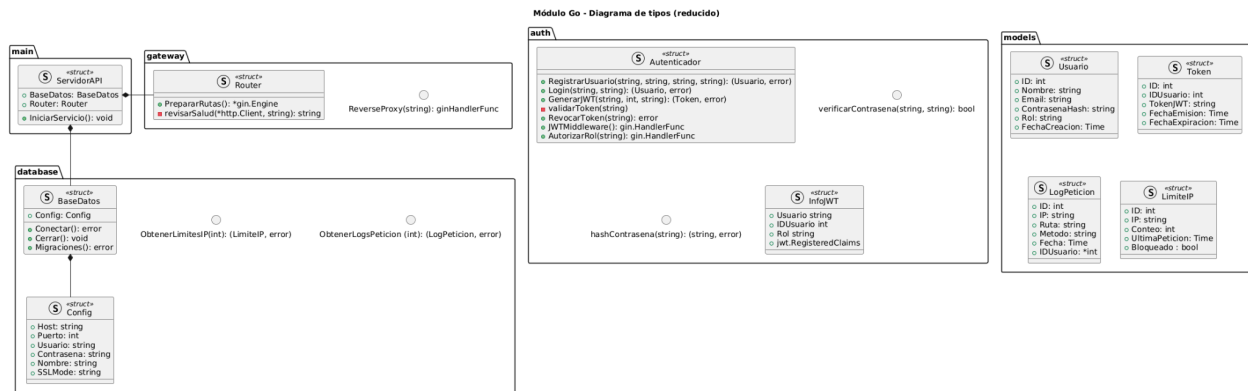
## Configuración del servicio en Docker

Dentro del archivo docker-compose.yml, este microservicio tiene la siguiente configuración por defecto:

- Hostname: sigefve\_go
- Puerto interno: 8080
- Puerto público: 8090

## Diagrama de tipos

Debido a que Go no es un lenguaje de programación orientado a objetos, sino orientado a interfaces y basado en objetos, no existe la definición de diagrama de clases aquí, sin embargo, es posible usar la notación de struct de UML para tratar de representar las estructuras que se utilizaron. Es importante mencionar que Go no permite la definición de métodos en las estructuras directamente, sino que estas se vinculan con la estructura fuera de su declaración, sin embargo, para que el diagrama sea más fácil de entender, los métodos se incluyeron dentro de las estructuras:



## Modelo de datos

La persistencia se maneja mediante PostgreSQL, como se mencionó en la sección de Stack tecnológico. El esquema utilizado es el siguiente:

Tabla: Usuario

Almacena la información de los usuarios.

| Campo           | Tipo         | Descripción                                                         |
|-----------------|--------------|---------------------------------------------------------------------|
| id              | serial PK    | Identificador único autoincremental.                                |
| nombre          | varchar(50)  | Nombre del usuario                                                  |
| email           | varchar(100) | Correo electrónico de la cuenta, funciona como el nombre de usuario |
| contrasena_hash | varchar(255) | Contraseña encriptada                                               |
| rol             | varchar(20)  | "Administrador" o "Conductor"                                       |
| fecha_creacion  | timestamp    | Fecha de creación del usuario                                       |

Tabla: Token

Almacena los tokens del sistema.

| Campo | Tipo | Descripción |
|-------|------|-------------|
|-------|------|-------------|

|                  |           |                                                         |
|------------------|-----------|---------------------------------------------------------|
| id               | serial PK | Identificador único autoincremental.                    |
| id_usuario       | int FK    | Referencia a la tabla Usuario.                          |
| token_jwt        | text      | Token JWT generado (email, id y rol).                   |
| fecha_emision    | timestamp | Fecha en que se generó el token.                        |
| fecha_expiracion | timestamp | Fecha en que expira el token (para eliminación lógica). |

Tabla: LogPeticion

Bitácora de peticiones de los usuarios.

| Campo      | Tipo         | Descripción                                            |
|------------|--------------|--------------------------------------------------------|
| id         | serial PK    | Identificador único autoincremental.                   |
| ip         | varchar(45)  | IP desde donde se hizo la petición.                    |
| ruta       | varchar(150) | Endpoint al que se dirigió el usuario.                 |
| metodo     | varchar(10)  | Método de la petición (GET, POST, PATCH, PUT, DELETE). |
| fecha      | timestamp    | Fecha en que se hizo la petición.                      |
| id_usuario | int FK       | Referencia a la tabla Usuario.                         |

Tabla: LimitIP

Registro de IP para limitar el número de peticiones que puede hacer.

| Campo | Tipo        | Descripción                          |
|-------|-------------|--------------------------------------|
| id    | serial PK   | Identificador único autoincremental. |
| ip    | varchar(45) | IP desde donde se hizo la            |

|                 |           |                                                  |
|-----------------|-----------|--------------------------------------------------|
|                 |           | petición.                                        |
| conteo          | int       | Cantidad de peticiones que ha hecho la IP.       |
| ultima_peticion | timestamp | Última vez que se hizo una petición desde la IP. |
| bloqueado       | boolean   | Indicador para saber si la IP está bloqueada.    |

## Especificación de la API RESTful

A continuación se detallan los contratos de los endpoints implementados en router.go.

### *Endpoints públicos*

#### **POST /login**

Permite iniciar sesión al sistema con las credenciales del usuario (correo electrónico y contraseña).

Cuerpo (JSON):

- username (string): Usuario o email
- password (string): Contraseña del usuario

Códigos de retorno:

- 200 OK: Login exitoso. Retorna: token (JWT), user\_id, rol
- 400 Bad Request: JSON inválido. Retorna: error
- 401 Unauthorized: Usuario o contraseña incorrectos. Retorna: error
- 500 Internal Server Error: No se pudo generar el token. Retorna: error

#### **GET /health**

Verifica el estado de salud del gateway y los microservicios conectados (Python y Java).

Códigos de retorno:

- 200 OK: Retorna el estado general del sistema y de cada servicio:
  - status: "ARRIBA", "PARCIAL" o "CAÍDO"
  - services: Estado individual de Go, Python y Java
  - timestamp: Fecha y hora de la verificación

### *Endpoints protegidos (cualquier usuario)*

A continuación, se listan los endpoints que requieren de haber iniciado sesión para ingresar (el token que genera el endpoint POST /login debe ir en un header de autenticación,

específicamente Bearer Token) y que pueden ser visibles para cualquier tipo de usuario, sin importar su rol:

**ANY /java/\*path**

Proxy inverso hacia el microservicio Java. Reenvía cualquier método HTTP (GET, POST, PUT, DELETE, etc.) hacia <http://sigefve-java:8585>.

Parámetros:

Dependen del endpoint específico del microservicio Java.

Códigos de retorno:

Dependen de la respuesta del microservicio Java.

**ANY /python/\*path**

Proxy inverso hacia el microservicio Python. Reenvía cualquier método HTTP (GET, POST, PUT, DELETE, etc.) hacia <http://sigefve-python:5001>.

Parámetros:

Dependen del endpoint específico del microservicio Python.

Códigos de retorno:

Dependen de la respuesta del microservicio Python.

***Endpoints protegidos (solo para el rol Administrador)***

Además de requerir el token en el header de la petición, se validará que el rol del usuario sea "Administrador" (no es necesario editar la petición con un parámetro adicional, pues el token almacena el rol del usuario):

**GET /admin/rate-limit**

Consulta el estado de los límites de peticiones por IP.

Parámetros (opcional):

- limit (default: 50, mínimo 1, máximo 100): Número de registros a obtener.

Códigos de retorno:

- 200 OK: Retorna lista de límites IP.
- 500 Internal Server Error: Error consultando LimitesIP. Retorna: error, detalle.

**GET /admin/logs**

Consulta el registro (logs) de peticiones al sistema.

Parámetros (opcional):

- limit (default: 100, mínimo 1, máximo 500): Número de registros a obtener.

Códigos de retorno:

- 200 OK: Retorna lista de logs de peticiones.
- 500 Internal Server Error: Error consultando LogsPeticion. Retorna: error, detalle.